

SKILL WEEK10

TASK1: To register handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t interrupt_count = 0;

void handle_sigint(int signal)
{
    interrupt_count++;

    printf("\nSIGINT received! Interrupt count = %d\n",
        interrupt_count);

    if (interrupt_count >= 3)
    {
        printf("Received SIGINT 3 times. Exiting...\n");
        exit(0);
    }
}

int main()
{
    struct sigaction action;

    action.sa_handler = handle_sigint;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    printf("Program started.\n");
    printf("Press Ctrl+C to send SIGINT.\n");
```

```
int main()
{
    struct sigaction action;

    action.sa_handler = handle_sigint;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    printf("Program started.\n");
    printf("Press Ctrl+C to send SIGINT.\n");
    printf("Press Ctrl+C three times to exit.\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```



```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigint(int signal)
{
    if (foreground_pid > 0)
    {
        printf("\nShell: Forwarding SIGINT to foreground job...\n");

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("\nShell: No foreground job running.\n");
    }
}

int main()
{
    struct sigaction action;

    action.sa_handler = handle_sigint;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    printf("Simple shell started.\n");
```

```

        perror("fork");
        return 1;
    }

    if (foreground_pid == 0)
    {
        /* Child process */

        signal(SIGINT, SIG_DFL);

        printf("Foreground job started. PID = %d\n",
               getpid());

        while (1)
        {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }
    else
    {
        /* Parent / shell */

        printf("Shell PID = %d\n", getpid());
        printf("Foreground job PID = %d\n",
               foreground_pid);

        waitpid(foreground_pid, NULL, 0);

        foreground_pid = -1;

        printf("Foreground job terminated.\n");
    }

    return 0;
}

```

```

laasya@laasya:~/signal_lab$ nano shell_sigint.c
laasya@laasya:~/signal_lab$ gcc shell_sigint.c -o shell_sigint
laasya@laasya:~/signal_lab$ ./shell_sigint
Simple shell started.
Shell PID = 454
Foreground job PID = 455
Foreground job started. PID = 455
Foreground job running...
Foreground job running...
Foreground job running...
^C
Shell: Forwarding SIGINT to foreground job...
Foreground job terminated.
laasya@laasya:~/signal_lab$ |

```

TASK3 : To capture SIGTSTP, suspend processes, update job table, preserve state, resume later, and test suspension workflow.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigstsp(int signal)
{
    if (foreground_pid > 0)
    {
        printf("\nShell: Suspending foreground job...\n");

        kill(foreground_pid, SIGTSTP);
    }
    else
    {
        printf("\nShell: No foreground job.\n");
    }
}

int main()
{
    struct sigaction action;

    action.sa_handler = handle_sigstsp;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGTSTP, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    printf("Shell started.\n");
```

```

    printf("Foreground job PID = %d\n",
           foreground_pid);

    int status;

    waitpid(foreground_pid,
            &status,
            WUNTRACED);

    if (WIFSTOPPED(status))
    {
        printf("Job state: STOPPED\n");
        printf("Job is suspended. PID = %d\n",
               foreground_pid);

        printf("\nTo resume the job, open another terminal and run:\n");
        printf("kill -CONT %d\n", foreground_pid);

        waitpid(foreground_pid,
                &status,
                WUNTRACED);

        if (WIFCONTINUED(status))
        {
            printf("Job state: CONTINUED\n");
        }
    }
    else if (WIFEXITED(status))
    {
        printf("Job state: TERMINATED\n");
    }
}

return 0;
}
|

```

```

laasya@laasya:~/signal_lab$ nano shell_sigstsp.c
laasya@laasya:~/signal_lab$ gcc shell_sigstsp.c -o shell_sigstsp
laasya@laasya:~/signal_lab$ ./shell_sigstsp
Shell started.
Foreground job PID = 473
Foreground job started. PID = 473
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
^Z
Shell: Suspending foreground job...
Job state: STOPPED
Job is suspended. PID = 473

To resume the job, open another terminal and run:
kill -CONT 473
|

```

TASK4: To create process groups, assign group IDs, transfer terminal control, restore ownership, coordinate signals, and validate behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t child;
    pid_t shell_pgid;

    shell_pgid = getpgrp();

    printf("Shell PID: %d\n", getpid());
    printf("Shell process group: %d\n", shell_pgid);

    child = fork();

    if (child < 0)
    {
        perror("fork");
        return 1;
    }

    if (child == 0)
    {
        /* Child creates a new process group */

        setpgid(0, 0);

        printf("\nChild PID: %d\n", getpid());
        printf("Child process group: %d\n", getpgrp());

        printf("Child waiting for signals...\n");

        while (1)
        {
```

```

        perror("tcsetpgrp");
    }
    else
    {
        printf("Terminal control transferred to child group.\n");
    }

    sleep(5);

    /*
     * Take terminal control back.
     */
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("Terminal control restored to shell group.\n");
    }

    /*
     * Send SIGTERM to the complete child process group.
     */
    printf("Sending SIGTERM to child process group...\n");

    killpg(child, SIGTERM);

    waitpid(child, NULL, 0);

    printf("Child process terminated.\n");
    printf("Process group demonstration complete.\n");
}

return 0;
}

```

[illegible]